StudentPay AI — Security Self-Assessment Report

Permission-tier assumptions, current controls, risks, evidence and production recommendations

1. Assessment Scope

This assessment covers the supplied StudentPay AI HTML prototype. It is a client-side educational prototype with no custom backend. The report intentionally distinguishes implemented controls from controls that would be required in a production financial application.

2. Security Architecture

- Execution environment: user's web browser.
- Application trust boundary: the HTML/JavaScript application and the browser's localStorage origin.
- Persistent data boundary: localStorage; there is no remote database in the submitted code.
- Authentication boundary: local prototype account check performed in JavaScript.
- AI boundary: optional runtime object window.claude; unavailable runtime falls back to localAI().
- Payment boundary: absent. No payment or bank transaction is executed.

3. Current Security Controls

Control	Implemented?	Evidence / implementation
Password not stored in plaintext	Yes (prototype)	Per-user random salt and SHA-256 digest are stored.
Input validation	Yes	Registration and financial input validation functions.
HTML output escaping	Yes	esc() is used before user-controlled text is inserted into HTML.
Failed-login throttling	Yes (local)	Five failed attempts in 15 minutes trigger a local block.
Audit logging	Yes (local)	Selected actions are recorded in DB.audit.
Data minimization	Partial	No bank/payment credentials are collected.
Secure server-side authorization	No	No backend exists.
HTTPS enforcement	No app control	Depends on hosting environment.
Secure cookies / server sessions	No	No server session exists.
Encryption at rest	No	localStorage is not encrypted by the application.
Real provider verification	No	Scam shield is only a heuristic checklist.
Production-grade password KDF	No	SHA-256 is not a password-specific KDF.
Dependency/Supply-chain controls	No	No dependency manifest or

		automated scan is present.

4. Permission / Access Tier

Recommended competition classification for the current build: DEMO / EDUCATIONAL — LOCAL BROWSER DATA. The application should not be represented as a production financial service. A production version would require authenticated server-side access control and a clearly defined data-processing model.

5. Threats and Risk Rating

Threat	Likelihood	Impact	Risk	Mitigation / status
localStorage tampering	High	Medium	High	Expected limitation of client-side prototype; use server-side storage and authorization in production.
XSS through user-entered data	Medium	High	Medium	esc() is used for displayed text; production should add CSP and framework-level escaping.
Credential theft from same-origin script	Medium	High	High	No production protection; move authentication server-side and protect sessions.
Weak password hashing for production	Medium	High	High	Replace SHA-256 password hashing with Argon2id/bcrypt/scrypt server-side.
False exchange-rate assumptions	High	Medium	High	Clearly label rates as manual demo values; integrate a trusted rate source in production.
AI prompt/data leakage	Medium	High	Medium/High	Minimize context, define retention rules, sanitize inputs, and review provider/API privacy terms.
Fraud/scam false positive or negative	Medium	High	High	Treat checklist as educational warning only; integrate authoritative verification if productized.
Data loss on browser/site-data	Medium	Medium	Medium	Expected prototype behavior; provide

clearing				controlled server persistence in production.

6. Security Self-Assessment Checklist

☐ No real payment credentials are requested by the prototype.

☐ The UI states that it is an educational prototype and not financial advice.

☐ User-entered displayed text is escaped before HTML insertion.

☐ The prototype records selected security-relevant events.

☐ Authentication is demonstrative and must not be presented as production authentication.

☐ localStorage data should be treated as accessible to code running in the same browser origin.

☐ No production claim should be made about encryption, secure sessions, regulatory compliance or payment security.

☐ Before production use, conduct threat modeling, penetration testing, dependency review, server-side authorization review and privacy/legal review.

7. Required Production Security Upgrade Path

1. Introduce a backend API and database with server-side authorization.
2. Use HTTPS everywhere and secure, HttpOnly, SameSite session cookies or another vetted session mechanism.
3. Use Argon2id/bcrypt/scrypt for password storage with appropriate parameters.
4. Move all sensitive business rules and validation to the server; treat client values as untrusted.
5. Replace localStorage persistence for sensitive data with controlled server storage.
6. Add CSRF protection where cookie-authenticated state-changing requests are used.
7. Add rate limiting, account lockout policies, MFA where appropriate, and centralized audit logging.
8. Integrate a trusted exchange-rate source and display source/timestamp.
9. Integrate regulated payment/remittance services rather than implementing money movement directly.
10. Apply CSP, dependency pinning/scanning, secret management and regular security testing.
11. Define AI data retention, user consent, prompt-injection handling and sensitive-data filtering.

8. Final Security Statement for Judges

StudentPay AI is intentionally submitted as an educational prototype. Its security design demonstrates validation, local password hashing, basic login throttling, output escaping, local audit events and data minimization within the constraints of a browser-only prototype. It does not claim production

financial security or real payment processing. The absence of a backend is a documented scope limitation, not a hidden feature.